

Lesson 05 — Agents, Subagents, Commands and Skills

Including Exercise 02 — AI Agent Debate

Dr. Yoram Segal

© All rights reserved to Dr. Yoram Segal

מילות מפתח

סוכן (Agent) • Subagent • פקודה (Command) • מיומנות (Skill) • כלים (Tools) •
• חלון הקשר (Context Window) • הנדסת קונטקסט (Context Engineering) • מודל •
• Write/Select • תקשורת בין-תהליכית (IPC) • ששן (Session) • התחברות בלוגין מול •
• API Key • Claude CLI • Gatekeeper • Watchdog • אורקסטרציה של סוכנים •
• שיפוט דיבייט • JSON כפרוטוקול תקשורת • UV (virtual environment) • שכבת SDK •
• לוגים מובנים •

1 פתיחה — מהיכן אנחנו ממשיכים

ד"ר יורם סגל פתח את השיעור בציון כי החלק הראשון של הקורס שימש כהקדמה ליצירת מכנה משותף בין הסטודנטים: **טוקנים**, Context Windows, מבנה המחשב ומבנה הסוכן. השיעור הנוכחי מסמן את הכניסה הרשמית לעולם הסוכנים, ובסיומו תינתן המטלה הבאה — תרגיל 02 — אשר יבוצע באמצעות סוכנים.

הנחת המוצא: הסטודנטים כבר מכירים את אבני היסוד —

— **מודל LLM** — לב הסוכן.

— **Context Window** — חלון ההקשר המצטבר בכל ששן.

— **כלים (Tools)** — אוסף תוכנות / APIs שהסוכן יכול להפעיל.

— **RAG** — בסיס נתונים וקטורי, ולמעשה מקרה פרטי של כלי.

2 הסוכן כתהליך — IPC ו-Process

הנקודה המארגנת של השיעור: **סוכן הוא תהליך (Process)**. ולכן:

— שני סוכנים = שני תהליכים שרצים במקביל (Multi-Process).

— שלושה סוכנים = שלושה תהליכים מקבילים, וכן הלאה.

— תקשורת בין סוכנים היא בדיוק **Inter-Process Communication** שלמדתם במערכות הפעלה —
Signals, FIFO, Queues, Sockets

ד"ר סגל הדגיש: כשאתם רוצים להריץ דיבייט בין שני סוכנים, אתם בעצם פותחים שני תהליכים ושולחים ביניהם הודעות. אתם המנהלים – אתם קובעים אם יש סנכרון, אינטרפטים, חסימות, וכו'.

תובנה מרכזית

המסוגלות שלכם לבנות סוכן רציני קשורה ישירות להבנת מערכות הפעלה ולתקשורת בין תהליכים. כלי הכיתה הוא Claude CLI, אך אותו רעיון קיים ב-Codex של OpenAI וב-Gemini של Google.

3 שתי דרכי התחברות ל-LLM

ד"ר סגל הציג שתי שיטות עבודה מול ספק המודל:

3.1 התחברות בלוגין (Login)

- התחברות לחשבון Claude / Google / OpenAI – בדרך כלל בעזרת Gmail.
- **יתרון:** bundle של טוקנים מראש, אין צורך לעקוב אחרי כל בקשה; כל בקשה מורגשת חיימית עד גמר המכסה.
- **חסרון:** משלמים את העלות הגלובלית של החבילה, גם אם בפועל ניצול הטוקנים נמוך.

3.2 התחברות באמצעות API Key

- מחוברים לספק בעזרת מפתח שמוטמע בקוד או ב-Environment Variable.
- **יתרון:** משלמים פר טוקן בלבד – בשימוש חכם זול מאוד.
- **חסרון:** צריך לנהל כלכלת טוקנים. מומלץ להגדיר אצל הספק **הגבלת תקציב** שתחסום בקשות אחרי סכום מסוים.
- **Gatekeeper** – ד"ר סגל ציפה שכל קוד יכלול שכבת שומר שמודדת ומגבילה צריכת טוקנים וגם יכול לבצע משימות נוספות כמו עזרה באבטחת סייבר.

3.3 מודלים חלופיים בעלות נמוכה

- **GLM (סיני, דרך Z.AI)** – מחיר של חודש Claude שווה לשנה שלמה של GLM. אפשר להחליף את המודל מתחת ל-Claude CLI ולהשאיר את חוויית העבודה זהה.
- Codex של OpenAI – אופציה חיימית סבירה.
- Gemini CLI – נוח, אך החינמי נחסם מהר.

הפקודה שחוסכת אישורים אינסופיים

בעבודה בתוך פרויקט ידוע ומבודד, אפשר להפעיל את הסוכן עם דגל ה-Yolo (לדוגמה dangerously-skip-permissions -ב-Claude CLI; ב-Gemini זה -yolo). הסוכן יפעל חופשי בתוך התיקייה ללא בקשות אישור חוזרות. **להפעיל אך ורק כשאתם בטוחים בגבולות הפעולה.**

4 Session, Project וסשנים גלובליים מול לוקליים

- **Session** – סדרת שאלות-תשובות הנמשכת עד איפוס Context Window.
- כל סשן **מצטבר** – ככל שמדברים יותר, חלון ההקשר גדל. סשן חדש מאפס.
- תיקיית claude. קיימת בשתי רמות:
 - **גלובלית** – לכל הפרויקטים.
 - **לוקלית** – בכל פרויקט בנפרד.
- תחת projects/ נשמרים סשנים בפורמט **JSONL** (JSON Lines) – כל שורה היא רשומת JSON. היתרון: שגיאה בשורה אחת אינה שוברת את הקובץ כולו.
- **משמעות מעשית:** ב-Context Engineering ניתן יהיה לקרוא, לערוך ולנהל סשנים ישירות מתוך קוד Python.

5 Context Windows Engineering – Select/Write model

- ד"ר סגל הציג את ההבדל בין **Prompt Engineering** ל-**Context Engineering**:
- ל-Transformer **אין זיכרון** – בכל אינטראקציה אנו מעלים מחדש את כל ההקשר.
 - **Write** – סינון/הוצאה (extraction) של מידע מתוך חלון ההקשר ושמירתו בצד (לדוגמה: מסמך גדול שכבר תורגם, ניתן לפנות אותו).
 - **Select** – החזרת מידע ששמרנו בצד אל החלון, רק כאשר הוא נדרש מחדש.

המקצוענות שאליה אנו מתכוונים

"אתם לא משתמשי ChatGPT פשוטים – אתם מנהלים כלכלת טוקנים. אתם בוחרים מה להכניס ומה להוציא מ-Context Window, וכך יוצרים תוצרים אמינים." – ד"ר יורם סגל

5.1 מנגנון Cache של הספקים

- לכל ספק (Claude, Gemini, OpenAI) יש מנגנון **Cache** פנימי שעובד ברמה הוקטורית / embedding.
- שאלה החוזרת עם אותו ההקשר עולה משמעותית פחות – לעיתים עשירית מחיר.

- אנחנו (כמשתמשי Inference) לא ניגשים ישירות ל-embedding הפנימי שלהם – רק הספק יכול לכתוב לתוך ה-Cache.

6 ההיררכיה: Agent → Subagent → Skill → Command

ד"ר סגל בנה את ההיררכיה צעד-צעד:

6.1 Command – פקודה

- פרומפט שמור עם שם, מופעל באמצעות />שם הפקודה<.
- הוא בעצם **copy-paste** – תוכן הפרומפט נכתב פעם אחת, מופעל אינסוף פעמים.
- **Command מופעל אך ורק על-ידי המשתמש.**
- הפקודה נשמרת כקובץ markdown בתיקיית הפקודות (בין אם מדובר בתיקיית הפקודות הגלובלית ובין אם מדובר בתיקיית הפקודות של Claude המקומית בתוך הפרויקט), בעלת שם זהה לשם הפקודה. הקריאה לפקודה מתוך ה-CLI היא באמצעות ./.

6.2 Skill – מיומנות

- גם Skill הוא פרומפט שמור, אך **הסוכן עצמו** יכול לאתר אותו ולהפעיל אותו.
- הסוכן מחליט באיזה Skill להשתמש לפי **Description** שבראש הקובץ – לכן ה-Description הוא הרכיב הקריטי ביותר ב-Skill.
- Skill הוא **תיקיה** המכילה skill.md (האינטרפייס) וקבצי Python משלימים – כלומר, ה-Skill תומך ב-**Tools**: אפשר לערב קוד שמבצע את העבודה החשובית הכבדה במקום לבזבז טוקנים.

חיסכון בטוקנים מול גמישות

טוקנים בלבד: גמיש וסבלני (עוצב לזהות "א" בכתב יד או בדפוס), אך יקר.
קוד (Regex / Python): עולה כמעט אפס טוקנים, אך לא גמיש – Regex לא יזהה ניואנסים.
האמנות: למצוא את האיזון. Skill מאפשר בדיוק את זה: מעטפת אנושית מעל קוד דטרמיניסטי.

6.3 Agent – סוכן

- **אוטונומי** – אינו "one-shot". הסוכן מקבל משימה, מבצע, **בוחר את התוצאה ויכול להתווכח איתה.**
- יודע לבחור Skills **אוטומטית** לפי תיאוריהם, וגם להפעיל Skills ספציפיים שצוינו במפורש.
- יכול להתווכח עם Skill שלא ביצע מספיק טוב ("חצי מהמסמך לא תורגם, בצע שוב ובדוק שכל מילה תורגמה").

6.4 Subagent

- עוד Agent שמופעל על-ידי האב. **קופסה שחורה** – האב לא רואה את התהליך הפנימי.
- יוצר מבנה היררכי: **מאסטר** (Agent) שמפרק משימה לתתי-משימות, ו-Subagents שמתמחים – חיפוש מקורות, כתיבה יפה, תרגום, וכו'.
- **כל** Subagent **מוסיף** Context Window **נפרד** – מאפשר שיחות ממוקדות לכל תפקיד, אך מעלה משמעותית את עלות הטוקנים. זה המחיר של אורקסטרציה.
- ה-Subagent **מת** בסיום המשימה – אינו ממשיך לפעולה הבאה.

מבנה ההפעלה ב-Claude CLI

פותחים את ה-CLI, מקלידים `/agents` (או דומה), ובוחרים `Create New Agent`. אפשר להגדיר אותו ב-**רמה גלובלית** (לכל פרויקט) או **לוקלית** (פרויקט נוכחי בלבד). הסוכן יכול לכתוב את ההוראות לעצמו (Generate) או לקבל אותן מאיתנו.

7 Router-Skill – ניהול חכם של רשימת Skills

- ד"ר סגל ציין שעם הזמן נצברים עשרות Skills גלובליים, וכל אחד מוסיף את ה-Description שלו ל-System Prompt – דבר שמבזבז טוקנים יקרים.
- הפתרון האישי שלו: **Router-Skill**.
- נטען תמיד מ-System Prompt.
 - כולל קוד Python שבודק האם זו השאלה הראשונה בסשן.
 - אם כן – קורא את משפט הפתיחה ובוחר רק את ה-Skills הרלוונטיים לדומיין הספציפי.
 - רק אותם נטענים ל-System Prompt, שאר המיומנויות נשמרות בצד.
- זוהי דוגמה מובהקת ל-Context Engineering מתקדם: ניהול ממוקד מטרה של מה נכנס לחלון ההקשר.

הערת מימוש מעשי

המימוש המעשי מחייב שימוש במנגנון hooks של Claude שלא הוזכר בשיעור. הסיבה היא שניהול ה-Skills שישמשו את ה-Session בתוך ה-System Prompt מוגדר בקובץ ה-Setup של Claude, ולכן מחייב אתחול מחדש של Claude באמצעות סקריפט חיצוני.

8 תרגיל 02 – דיבייט בין שני סוכני AI

8.1 המטרה

לבנות ויכוח בין שני סוכנים בפיקוח של סוכן שלישי. הארכיטקטורה הנדרשת:

– **סוכן אבא (שופט)** – אורח-מנחה, מקבל החלטות.

– **סוכן בעד** – מתמחה בעמדה אחת.

– **סוכן נגד** – מתמחה בעמדה הנגדית.

8.2 נושא הויכוח בין הסוכנים

חופשי לחלוטין – מי טובה יותר ברצלונה או ריאל מדריד, האם יש אלוהים או אין, איזו דיאטה עדיפה, אילו דגים טעימים יותר (מים מתוקים מול מלוחים), וכו'.

8.3 התניות מחייבות

1. **דו-שיח מכבד** – אחד מדבר, מסיים, מאזין; מגבילים בזמן או בכמות מילים.
2. **קונטרדיקציה אמיתית** – לכל סוכן Skill שונה משל חברו, כדי שהויכוח לא ימוטט את עצמו. סוכנים נוטים "לרצות" – דאגו שלא יסכימו אחד עם השני אוטומטית.
3. **לפחות 10 Pings** (טיעון → טיעון-נגד) לכל צד.
4. **התייחסות הדדית** – כל סוכן חייב להתייחס לטיעוני יריבו, לא לדבר במקביל.
5. **אסמכתאות מהאינטרנט** – tool של חיפוש באינטרנט הוא חובה.
6. **תפקיד האבא: אסור** שיהיה תיקו. הוא חייב להחליט מי זכה, ולנמק את הציון. הקריטריון הוא **כושר שכנוע**, לא נכונות עובדתית – כמו המשחק "האמת היא שקר" בטלוויזיה.
7. **הויכוח עובר דרך האבא** – לא ישירות בין הילדים. כל הודעה: ילד → אבא → ילד.
8. **פורמט תקשורת** – JSON. מובנה תבניתית, ניתן לניטור ובדיקה, וחוסך טוקנים.

8.4 מה לא לעשות

- **אסור תוצאת תיקו בין הצדדים** – האב חייב להכריע.
- **אין ויכוח ללא שימוש ב-LLM** – הויכוח חייב להיות אמיתי, לא טקסט מומצא מקוד Python.
- **אין שפה בוטה** – Politically Correct ומכבדת.
- **אסור להגיש דרך Claude CLI בלבד** – חייבים לעבוד עם קוד Python שמפעיל את הסוכנים.

8.5 שלבי בנייה מומלצים

ד"ר סגל הציע גישה מדורגת:

1. **שלב ידני (בטרמינל)** – לפתוח שני Claude CLI (או GPT + Gemini), להגדיר לכל אחד תפקיד, ולנהל ויכוח ידני. רק כדי להבין את התופעה.
2. **שלב ביניים** – Claude CLI ב-Command שמפעיל את האב, שמפעיל את הילדים.
3. **שלב סופי** – קוד Python ראשי שמנהל את שלושת התהליכים.

8.6 שיקולי הנדסה – חובה

- **Timeouts** בכל בקשה – סוכן עלול להיתקע.
- **Watchdog עם keep-alive** – חיוני בכל פרויקט סוכנים אוטונומי. אם תהליך נופל, יש להרוג ולהפעיל מחדש.
- **Object Oriented (OOP)** – הארכיטקטורה של התוכנה חייבת להיות מבוססת תכנון מבנה המבוסס על מחלקות והורשה בין מחלקות. אסור לשכפל קוד; אם יש קוד משותף יש לייצר מחלקה משותפת. יש לצרף שרטוט הארכיטקטורה של פריסת המחלקות והקשרים ביניהן.
- **TDD** ובדיקות יחידה.
- **Ruff + Linter**.
- **ללא פרמטרים hardcoded** – הכל בקובץ תצורה.
- **בדיקת Cyber** – לא להשאיר API Keys ב-Git. רק `env.example`, ו-`env`. נכנס ל-`gitignore`.
- **Gatekeeper** – שכבת חסימה כלכלית וצריכתית.
- **שכבת SDK** – מודולים עצמאיים מתחת לממשק (Terminal/CLI/UI/API) שמאפשרת לסוכן AI לדבג את עצמו.
- **לוגים מובנים** – חבילה מוכנה, FIFO המוגדרים בקובץ הקונפיגורציה – למשל 20 קבצים, כאשר כל קובץ מכיל עד 500 שורות.
- **תפעול מהטרמינל** – פרויקט זה חייב להכיל יכולת תפעול מתפריטים בסיסיים מתוך הטרמינל (תפעול באמצעות מקלדת). מי שרוצה יכול גם להכין GUI ולצרף תמונות מסך של התפעול, אך הבדיקה תהיה מבוססת תפעול באמצעות התפריט או באמצעות ה-SDK ישירות.

8.7 דרישות הגשה

- `README.md` מפורט עם תמונות מסך, פרומפטים, ודיאלוג מלא של סשן 1.
- שיחות באנגלית או בעברית (לא ערבית) – ד"ר סגל מבקש פורמט שהוא יכול לקרוא.
- **UV (virtual environment)** – מומלץ מאוד. הקובץ `pyproject.toml` צריך לאפשר הקמת סביבה מלאה אצל הבודק.

- `env.example`. בלבד, לא `..env`.
- **הגשה בזוגות בלבד**. כל אחד מבני הזוג חייב להגיש בנפרד ב-Moodle את קובץ ה-PDF של הקישור לרפוזיטורי. יש להקפיד להגיש קישור זהה לאותו הרפוזיטורי המשותף לשני חברי הקבוצה – ולשתפו עם המרצה או באופן ציבורי.
- **עבור מי שיש מגבלת תקציב**: אפשר להוריד מ-10 pings ל-5, ולציין זאת במפורש ב-README – ולא ייגרע מהציון.
- **ההגשה באמצעות GitHub**. לא לשכוח לשתף עם המרצה או באופן ציבורי (מומלץ ציבורי). הגשה שאין אליה גישה – נפסלת, ואין אפשרות להגיש מחדש!

9 נקודות חידוד מהדיון הכיתתי

- **תיקו (Tie)?** ממש לא. אפשר לתת ניקוד דיפרנציאלי (80% / 70%), אבל הכרעה חייבת.
- **האם השופט חייב לדעת את הנושא?** לא. השופט מבין רק את **חוקי המשחק** ושופט כושר שכנוע. דווקא טוב שהוא לא יודע – כך אינו מוטה.
- **הסכמות במהלך הוויכוח?** מותרות באירוע נקודתי, אבל אסור שסוכן אחד יסחף את חברו לכל אורך הוויכוח. האב חייב להתערב ולהזכיר את התפקיד.
- **שקרים בוויכוח?** מותר. הצד הנגדי אמור לתפוס אותם – זה חלק מכושר השכנוע.

10 תובנות סיכום

המסר המרכזי

המעבר מ-Prompt Engineering ל-Context Engineering הוא המעבר שהופך אתכם ממשתמשי ChatGPT למהנדסי סוכנים. אורקסטרציה של סוכנים, ניהול מודע של חלון ההקשר, ועיצוב היררכיה ברורה של Subagent, Agent, Skill, Command – אלו הכלים שיבדילו את התוצר שלכם מתוצר חובבני.